

Improved 2D U-Net for Hip MRI Prostate Segmentation

1 Overview

This project tackles binary prostate segmentation from 2D Hip MRI slices in the nifti format. It implements an **improved U-Net** that adds pre activation residual blocks, InstanceNorm, Dropout in 20% cases, and optional deep supervision to the classic U-Net encoder decoder structure.

The goal is to get accurate pixel-wise segmentation masks for the prostate which are evaluated by Dice similarity score. For visualisation, losses.png plots train/validation loss and dice.png plots validation dice score.

2 Algorithm

The architecture is as follows (with more comprehensive commenting in the code itself):

- **Encoder-decoder U-Net** with three down/upsampling stages and a bottleneck.
- **Residual pre-activation blocks** with InstanceNorm then LeakyReLU then Conv2D.
- **Optional dropout** between convolutions and a 1×1 projection on skip if channels change.
- **InstanceNorm** instead of BatchNorm for stability at small batch sizes.
- **Deep supervision** with auxiliary 1×1 heads from decoder stages that are upsampled and combined into the loss to improve gradient flow and regularisation.

The loss metrics are:

- **Combined loss** with $\lambda \times \text{BCE} + (1 - \lambda) \times \text{Dice}$ where λ defaults to 0.5.
- **Deep supervision weights** are set at default for main head at 1.0, auxiliary head 2 at 0.5, and auxiliary head 3 at 0.25 (trimmed).
- **Dice** coefficient is used on the main head.

Preprocessing was also done with the following methods:

- NIFTI parsing via provided code from Appendix B.
- Standardise each slice via Z-score with mean = 0 and std = 1.0: $z = \frac{(x - \mu)}{\sigma}$.

3 Performance

The final model achieves an ~98% Dice score on the TEST set, and 98.07% Dice on the validation set. Thus, we successfully reach and surpass the threshold of ≥ 0.75 Dice on test set. Though we have 10 epochs in the loop, an early stop is enforced when the validation Dice is > 0.98 , which occurred at 5 epochs (see end of this README for logs).

4 Data Layout x

The expected disk data layout is:

```
HipMRI_2D/ ├── keras_slices_train/ ├── keras_slices_validate/ ├── keras_slices_test/ ├──
keras_slices_seg_train/ ├── keras_slices_seg_validate/ ├── keras_slices_seg_test/
```

where HipMRI_2D should be placed on device ~/Desktop. This data is taken exactly from Rangpur. We use the provided splits from Rangpur to avoid leakage and ensure fair assessment. The validation set tracks model selection/early stopping for epochs once validation dice is >98%, and the final test set is used only **once** to report performance. This prevents any bleeding and false scores, and mirrors standard practice.

5 Files

- modules.py - model components (ResidualBlock, UpBlock, ImprovedUNet).
- dataset.py - Dataset + DataLoader generation, NIFTI parsing, z-score preprocessing.
- train.py - train/validate/test loops, plotting (losses.png, dice.png), model saving.
- predict.py — example inference & overlay creation (see usage below).
- README.md - this report.
- utils.py - optional helper/tester file to ensure files parse correctly (this is not needed).

6 Dependencies + Extras

python >= 3.10 torch >= 2.2 torchvision >= 0.17 (if installed, but not required here). Other versions may cause conflict. nibabel >= 5.2 numpy >= 1.26 matplotlib >= 3.8

For speed on NVIDIA GPUs, we use cudnn.benchmark=True. During training, the device is chosen based on the best out of 3: CUDA (NVIDIA), MPS (Apple Silicon), or CPU.

7 Running Code

1. Put the dataset at ~/Desktop/HipMRI_2D/ (or change ROOT in train.py).
2. run: python train.py
3. This outputs the best model, losses.png and dices.png.

The following (or something very similar) should print when ran:

```
Using external GPU (CUDA) Epoch 01/10 | train_loss=0.0785 | val_loss=0.0775 | val_dice=0.9762 | Saved
new best model (val_dice=0.9762) Epoch 02/10 | train_loss=0.0457 | val_loss=0.0689 | val_dice=0.9784 |
Saved new best model (val_dice=0.9784) Epoch 03/10 | train_loss=0.0424 | val_loss=0.0630 |
val_dice=0.9796 | Saved new best model (val_dice=0.9796) Epoch 04/10 | train_loss=0.0400 |
val_loss=0.0569 | val_dice=0.9794 Epoch 05/10 | train_loss=0.0386 | val_loss=0.0526 | val_dice=0.9807 |
Saved new best model (val_dice=0.9807) Validation Dice reached 0.98, stopping training early. TEST |
loss=0.0393 | dice=0.9836 Saved final model to improved_unet_last.pt Saved plots to losses.png and
dice.png
```

8 AI Declaration

Microsoft Copilot's autocomplete feature was used to assist with writing and debugging code. OpenAI's ChatGPT-5 was used to generate certain functions (these are declared in comments), including the main function in train.py, and pick_indices and save_overlay in predict.py.